

Towards AI

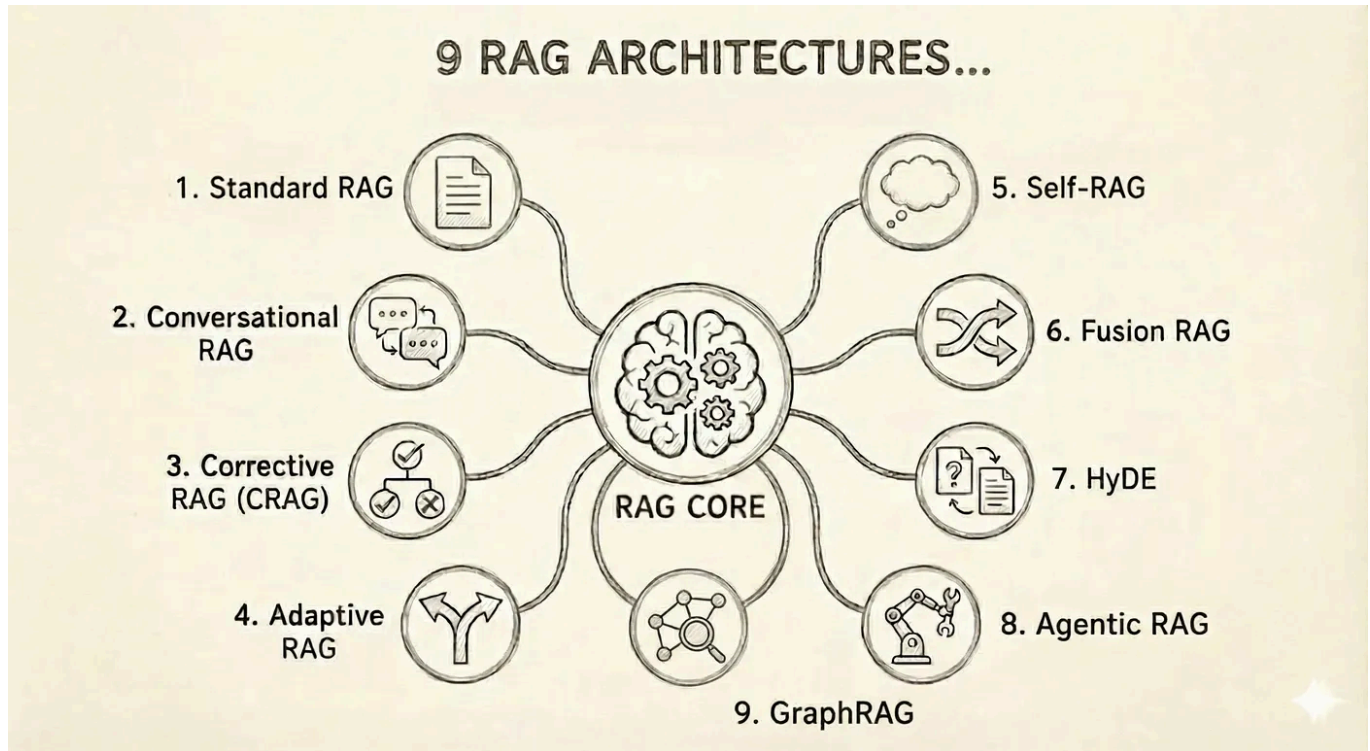


Photo by GEMINI

★ Member-only story

9 RAG Architectures Every AI Developer Must Know: A Complete Guide with Examples

Architectures beyond Naive Rag to build reliable production AI Systems



Divy Yadav

Follow

12 min read · Dec 19, 2025



1.1K



23



Your chatbot confidently told a customer your return policy is 90 days. It's

30 Welcome back. You are signed into your member account j*****@gmail.com.

That is the gap between a great demo and a real production system. Language models sound sure even when they are wrong, and in production that gets expensive fast.

This is why serious AI teams use RAG. Not because it is trendy, but **because it keeps models grounded in real information.**

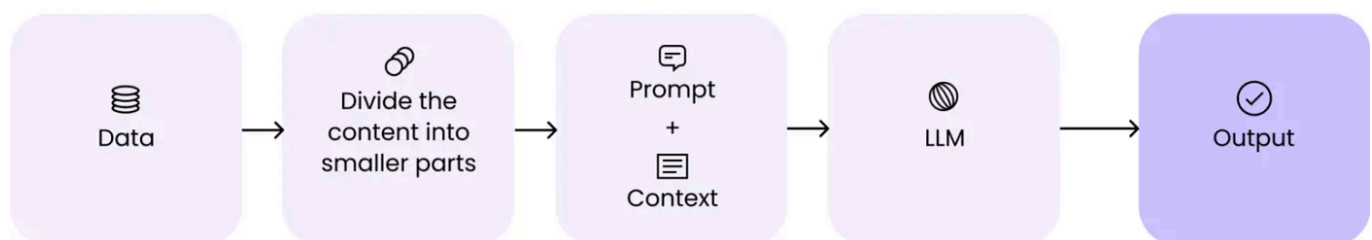
What most people miss is that there is no single RAG. **There are multiple architectures, each solving a different problem. Pick the wrong one, and you waste months.**

This guide breaks down the RAG architectures that actually work in production.

Let's start by Learning about Rag.

What Is RAG and Why Does It Actually Matter?

The RAG process



Welcome back. You are signed into your member account **jr•••••@gmail.com**.

Before we dive into architectures, let's get clear on what we're talking about.

RAG optimizes language model outputs by having them reference external knowledge bases before generating responses. Instead of relying purely on what the model learned during training, RAG pulls in relevant, current information from your documents, databases, or knowledge graphs.

Here's the process in practice.

- When a user asks a question, your RAG system **first retrieves relevant information from external sources** based on that query.
- Then it **combines the original question with this retrieved context** and sends everything to the language model.
- The model **generates a response grounded** in actual, verifiable information rather than just its training data.

The Real Problems that RAG Solves

Welcome back. You are signed into your member account jr.....@gmail.com.

The Knowledge Cutoff Problem



Your language model's training data has a cutoff date...

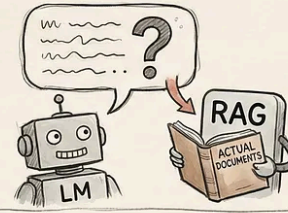


YESTERDAY,
LAST WEEK,
5 MINS AGO



RAG lets you access information from yesterday, last week, or five minutes ago.

The Hallucination Problem



Language models generate text that sounds authoritative...

DoorDash's guardrail system reduced hallucinations by 90% arXiv by ensuring responses were grounded in actual retrieved documents.

The Cost Problem



Retraining foundation models... costs hundreds of thousands...



RAG lets you incorporate new information without touching the model at all.

The Compliance Problem



In regulated industries... you need to know exactly where information came from.

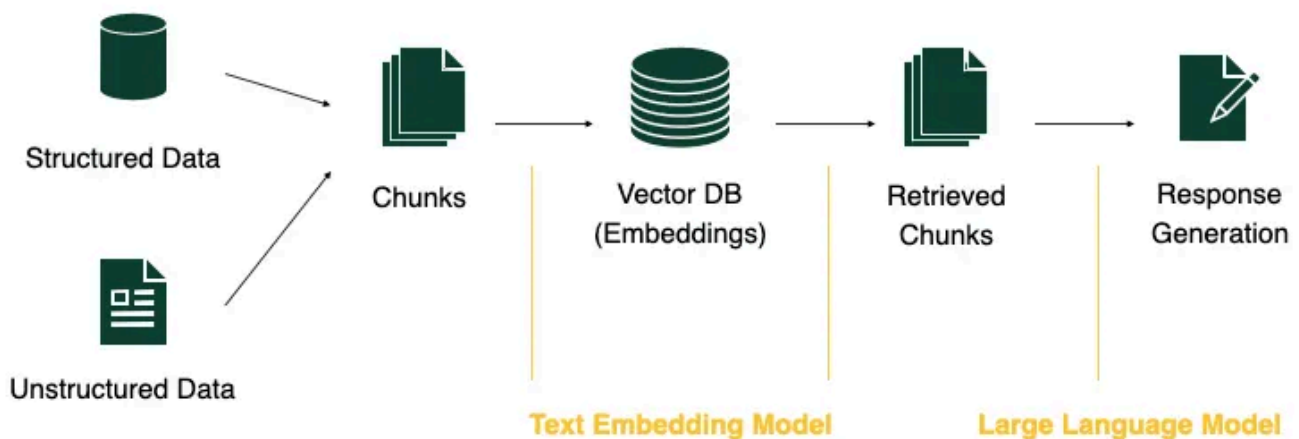
RAG systems can cite their sources, creating an audit trail...

Now let's get into the architectures... starting simple and building to the more sophisticated approaches.

Photo by Gemini

1. Standard RAG: Start Here

Simple RAG



Source: <https://www.bentoml.com/blog/building-rag-with-open-source-and-custom-ai-models>

Standard RAG is the “Hello World” of the ecosystem. It treats retrieval as a simple, one-shot lookup. It exists to ground a model in specific data without the overhead of fine-tuning, but it assumes your retrieval engine is perfect.

It is best suited for low-stakes environments where speed is more

in Welcome back. You are signed into your member account jr*****@gmail.com.

How it Works:

1. **Chunking:** Documents are split into small, digestible text segments.
2. **Embedding:** Each segment is converted into a vector and stored in a database (like Pinecone or Weaviate).
3. **Retrieval:** A user query is vectorized, and the “Top-K” most similar segments are pulled using Cosine Similarity.
4. **Generation:** These segments are fed to the LLM as “Context” to generate a grounded response.

Realistic Example: A small startup’s internal employee handbook bot. A user asks, “What is our pet policy?” and the bot retrieves the specific paragraph from the HR manual to answer.

Pros:

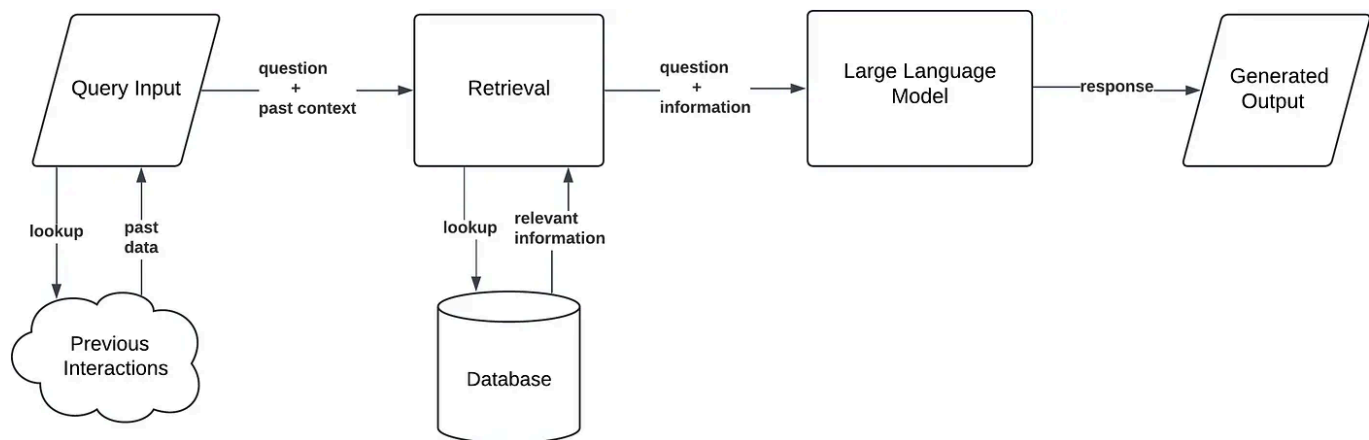
- Sub-second latency.
- Extremely low computational cost.
- Simple to debug and monitor.

Cons:

- Highly susceptible to “noise” (retrieving irrelevant chunks).
- No ability to handle complex, multi-part questions.
- Lacks self-correction if the retrieved data is wrong.

2. Conversational RAG: Adding Memory

Welcome back. You are signed into your member account jr.....@gmail.com.



Source: <https://humanloop.com/blog/rag-architectures>

Conversational RAG solves the problem of “context blindness.” In a standard setup, if a user asks a follow-up like “How much does it cost?”, the system doesn’t know what “it” refers to. **This architecture adds a stateful memory layer that re-contextualizes every turn of the chat.**

How it Works:

1. **Context Loading:** The system stores the last 5–10 turns of the conversation.
2. **Query Rewriting:** An LLM takes the history + the new query to generate a “Stand-alone Query” (e.g., “What is the price of the Enterprise Plan?”).
3. **Retrieval:** This expanded query is used for the vector search.
4. **Generation:** The answer is generated using the new context.

Realistic Example: A customer support bot for a SaaS company. The user says, “I’m having trouble with my API key,” and then follows up with, “Can you reset it?” The system knows “it” means the API key.

Pros:

- Provides a natural human-like chat experience

Welcome back. You are signed into your member account jr•••••@gmail.com.

-

Cons:

- **Memory Drift:** Irrelevant context from 10 minutes ago can pollute the current search

Open in app ↗



Medium



Search

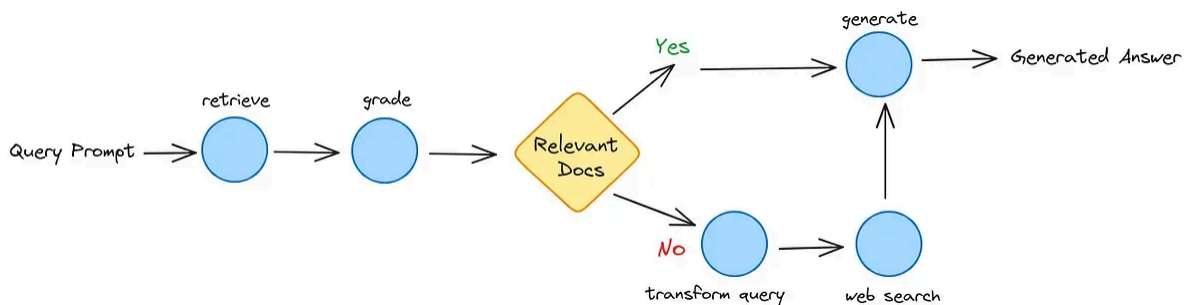


Write

13



3. Corrective RAG (CRAG): The Self-Checker



Source: <https://lancedb.com/blog/implementing-corrective-rag-in-the-easiest-way-2/>

CRAG is an architecture designed for high-stakes environments. It introduces a “**Decision Gate**” that evaluates the quality of retrieved documents before they reach the generator. If the internal search is poor, it triggers a fallback to the live web.

In internal benchmarks reported by teams deploying CRAG-style evaluators, hallucinations have been shown to drop as compared to naive baselines.

How it Works:

1. **Retrieval:** Fetch documents from your internal vector store.

2 **Evaluation:** A lightweight “Grader” model assigns a score (Correct

Welcome back. You are signed into your member account jr*****@gmail.com.

3. Trigger Gate:

- **Correct:** Proceed to the generator.
- **Incorrect:** Discard the data and trigger an external API (like Google Search or Tavily).

4. **Synthesis:** Generate the answer using the verified internal or fresh external data.

Realistic Example: A financial advisor bot. When asked about a specific stock price that isn't in its 2024 database, CRAG realizes the data is missing and pulls the live price from a financial news API.

Pros:

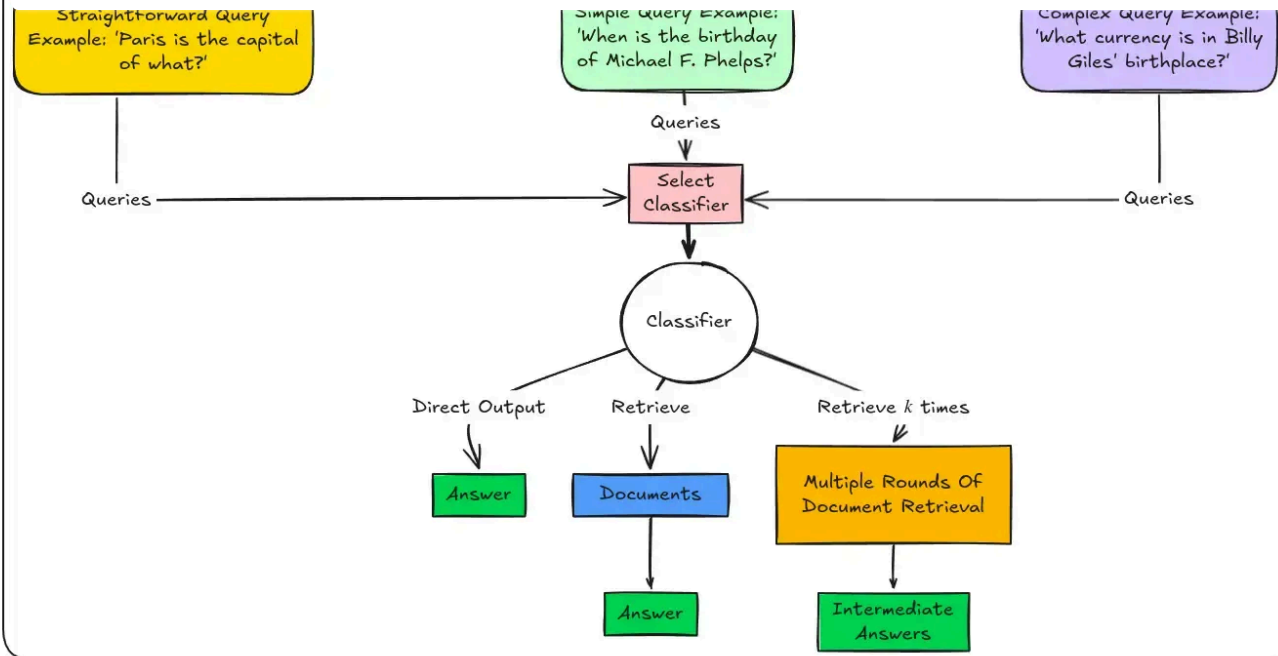
- Drastically reduces hallucinations.
- Bridges the gap between internal data and live, real-world facts.

Cons:

- Significant latency increase (adds 2–4 seconds).
- Managing external API costs and rate limits.

4. Adaptive RAG: Matching Effort to Complexity

Welcome back. You are signed into your member account jr.....@gmail.com.



Source: <https://www.analyticsvidhya.com/blog/2025/03/adaptive-rag-systems-with-langgraph/>

Adaptive RAG is the “efficiency champion.” It recognizes that not every query requires a bazooka. It uses a router to determine the complexity of a user’s intent and chooses the cheapest, fastest path to the answer.

How it Works:

1. **Complexity Analysis:** A small classifier model routes the query.
2. **Path A (No Retrieval):** For greetings or general knowledge the LLM already knows.
3. **Path B (Standard RAG):** For simple factual lookups.
4. **Path C (Multi-step Agent):** For complex analytical questions that require searching multiple sources.

Realistic Example: A university assistant. If a student says “Hello,” it responds directly. If they ask “When is the library open?”, it does a simple

search. If they ask “Compare the tuition of the CS program over the last 5 years”

Welcome back. You are signed into your member account jr.....@gmail.com.

Pros:

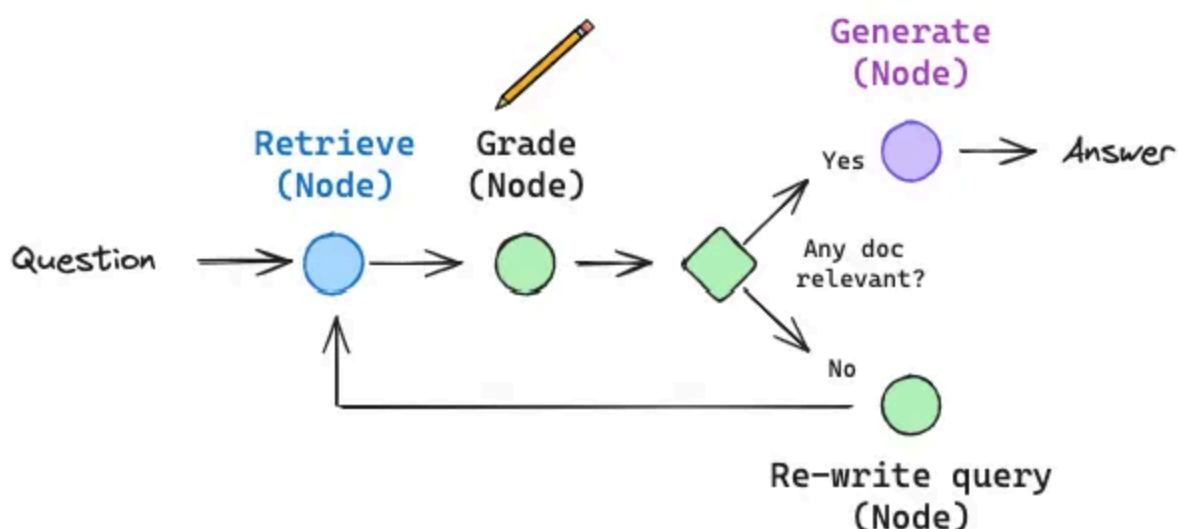
- Massive cost savings by skipping unnecessary retrieval.
- Optimal latency for simple queries.

Cons:

- Misclassification risk: If it thinks a hard question is easy, it will fail to search.
- Requires a highly reliable routing model.

5. Self-RAG: The AI That Critiques Itself

Self-Reflective RAG with LangGraph (Reflection and self-correction)



Welcome back. You are signed into your member account [jr*****@gmail.com](#).

Self-RAG is a sophisticated architecture where the model is trained to critique its own reasoning. **It doesn't just retrieve; it generates "Reflection Tokens" that serve as a real-time audit of its own output.**

How it Works:

1. **Retrieve:** Standard search triggered by the model itself.
2. **Generate with Tokens:** The model generates text alongside special tokens like `[IsRel]` (Is this relevant?), `[IsSup]` (Is this claim supported?), and `[IsUse]` (Is this helpful?).
3. **Self-Correction:** If the model outputs a `[NoSup]` token, it pauses, re-retrieves, and rewrites the sentence.

Realistic Example: A legal research tool. The model writes a claim about a court case, realizes the retrieved document doesn't actually support that claim, and automatically searches for a different precedent.

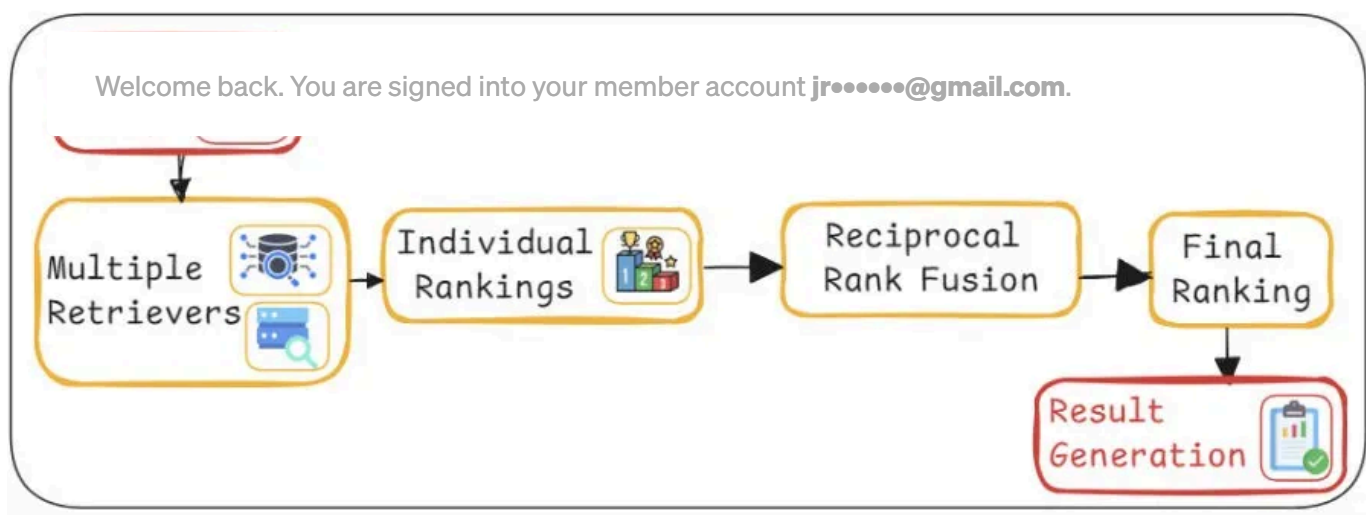
Pros:

- Highest level of factual "groundedness."
- Built-in transparency for the reasoning process.

Cons:

- Requires specialized, fine-tuned models (e.g., Self-RAG Llama).
- Extremely high computational overhead.

6. Fusion RAG: Multiple Angles, Better Results



Source: <https://bhavishyapandit9.substack.com/p/25-types-of-rag-part-1>

Fusion RAG addresses the “**Ambiguity Problem.**” Most users are bad at searching. Fusion RAG takes a single query and looks at it from multiple angles to ensure high recall.

How it Works:

1. **Query Expansion:** Generate 3–5 variations of the user’s question.
2. **Parallel Retrieval:** Search for all variations across the vector DB.
3. **Reciprocal Rank Fusion (RRF):** Use a mathematical formula to re-rank the results:
4. **Final Ranking:** Documents that appear high in multiple searches are boosted to the top.

Realistic Example: A medical researcher searching for “treatments for insomnia.” Fusion RAG also searches for “sleep disorder medications,” “non-pharmacological insomnia therapy,” and “CBT-I protocols” to ensure no relevant study is missed.

Pros:

- Exceptional recall (finds documents a single query would miss).

- Robust to poor user phrasing

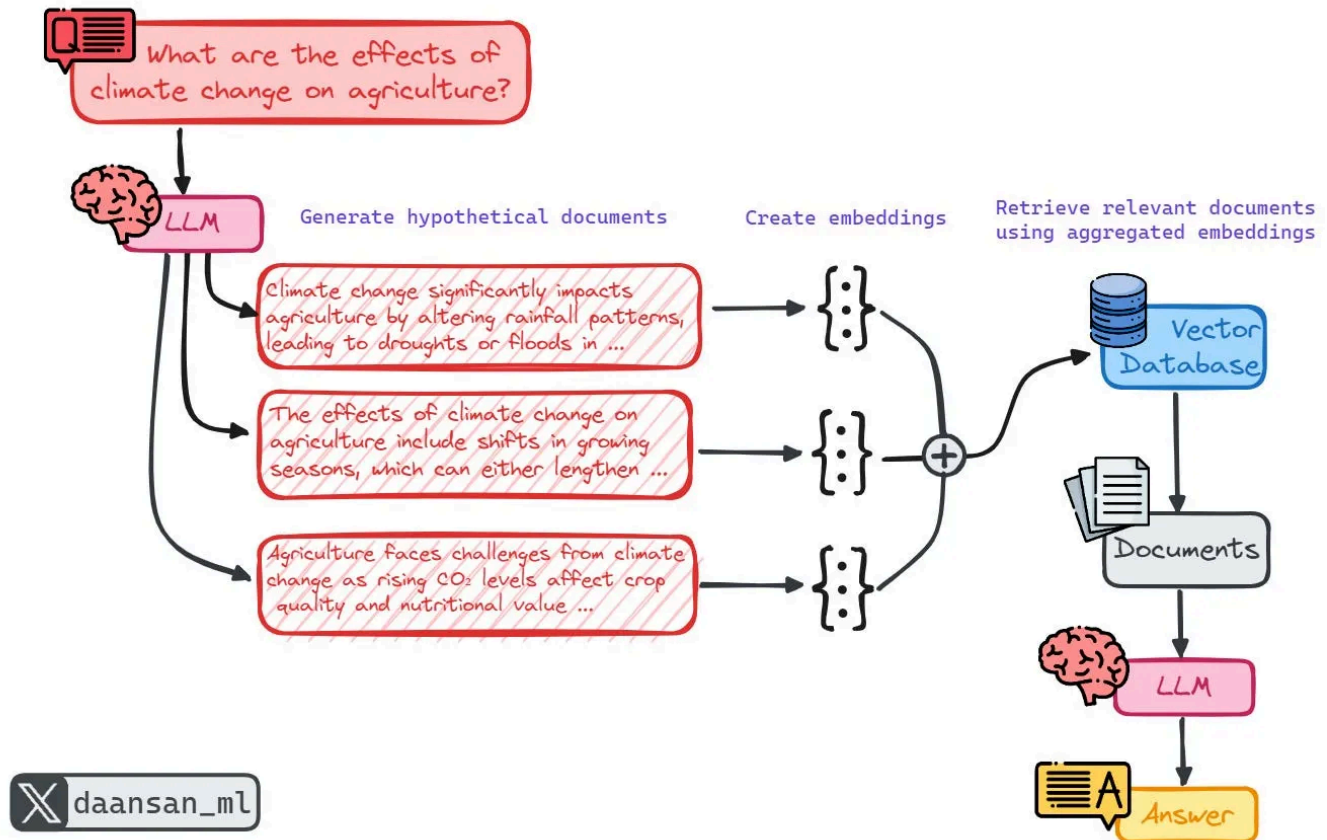
Welcome back. You are signed into your member account [jr.....@gmail.com](#).

Cons:

- Multiplies search costs (3x-5x).
- Higher latency due to re-ranking calculations.

7. HyDE: Generate the Answer, Then Find Similar Docs

Advanced RAG: HyDE



Source: <https://mlpills.substack.com/p/issue-85-advanced-retrieval-strategies>

HyDE is a counter-intuitive but brilliant pattern. It recognizes that "Questions" and "Answers" are semantically different. It creates a bridge between them by generating a "fake" answer first.

How it Works:

Welcome back. You are signed into your member account [jr.....@gmail.com](#).

1. **Hypothesize:** The LLM writes a *fake* (hypothetical) answer to the query.
2. **Embedding:** The *fake answer* is vectorized.
3. **Retrieval:** Use that vector to find real documents that look like the fake answer.
4. **Generation:** Use the real docs to write the final response.

Realistic Example: A user asks a vague question like “That one law about digital privacy in California.” HyDE writes a fake summary of CCPA, uses that to find the *actual* CCPA legal text, and provides the answer.

Pros:

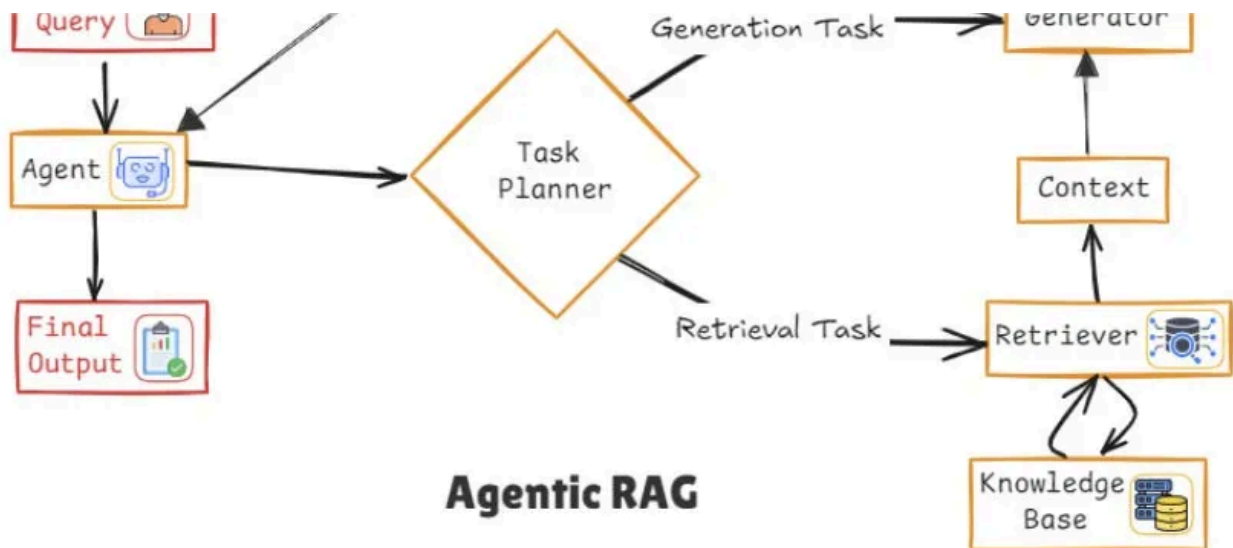
- Dramatically improves retrieval for conceptual or vague queries.
- No complex “agent” logic required.

Cons:

- Bias Risk: If the “fake answer” is fundamentally wrong, the search will be misled.
- Inefficient for simple factual lookups (e.g., “What is 2+2?”).

8. Agentic RAG: Orchestrating Specialists

Welcome back. You are signed into your member account jr.....@gmail.com.



Source: <https://bhavishyapandit9.substack.com/p/25-types-of-rag-part-1>

Instead of blindly fetching documents, it introduces an autonomous agent that plans, reasons, and decides *how* and *where* to retrieve information before generating an answer.

It treats information retrieval like research, not a lookup.

How it Works:

1. Analyze:

The agent first interprets the user query and determines whether it is simple, multi-step, ambiguous, or requires real-time data.

2. Plan:

It breaks the query into sub-tasks and decides a strategy.

For example, Should it do vector search first? Web search? Call an API? Ask a follow-up question?

3. Act:

The agent executes those steps by invoking tools such as vector databases, web search, internal APIs, or calculators.

4 Iterate:

Welcome back. You are signed into your member account [jr.....@gmail.com](#).

data, or validate sources.

5. Generate:

Once sufficient evidence is gathered, the LLM produces a grounded, context-aware final response.

Realistic Example:

A user asks:

“Is it safe for a fintech app to use LLMs for loan approvals under Indian regulations?”

Agentic RAG might:

1. Detect this is a regulatory + policy + risk question
2. Search RBI guidelines via web tools
3. Retrieve internal compliance documents
4. Cross-check recent regulatory updates
5. Synthesize a structured answer with citations and caveats

A traditional RAG would likely just retrieve semantically similar documents and answer once.

Pros:

- Handles complex, multi-part, and ambiguous queries
- Reduces hallucinations through verification and iteration
- Can access real-time and external data sources

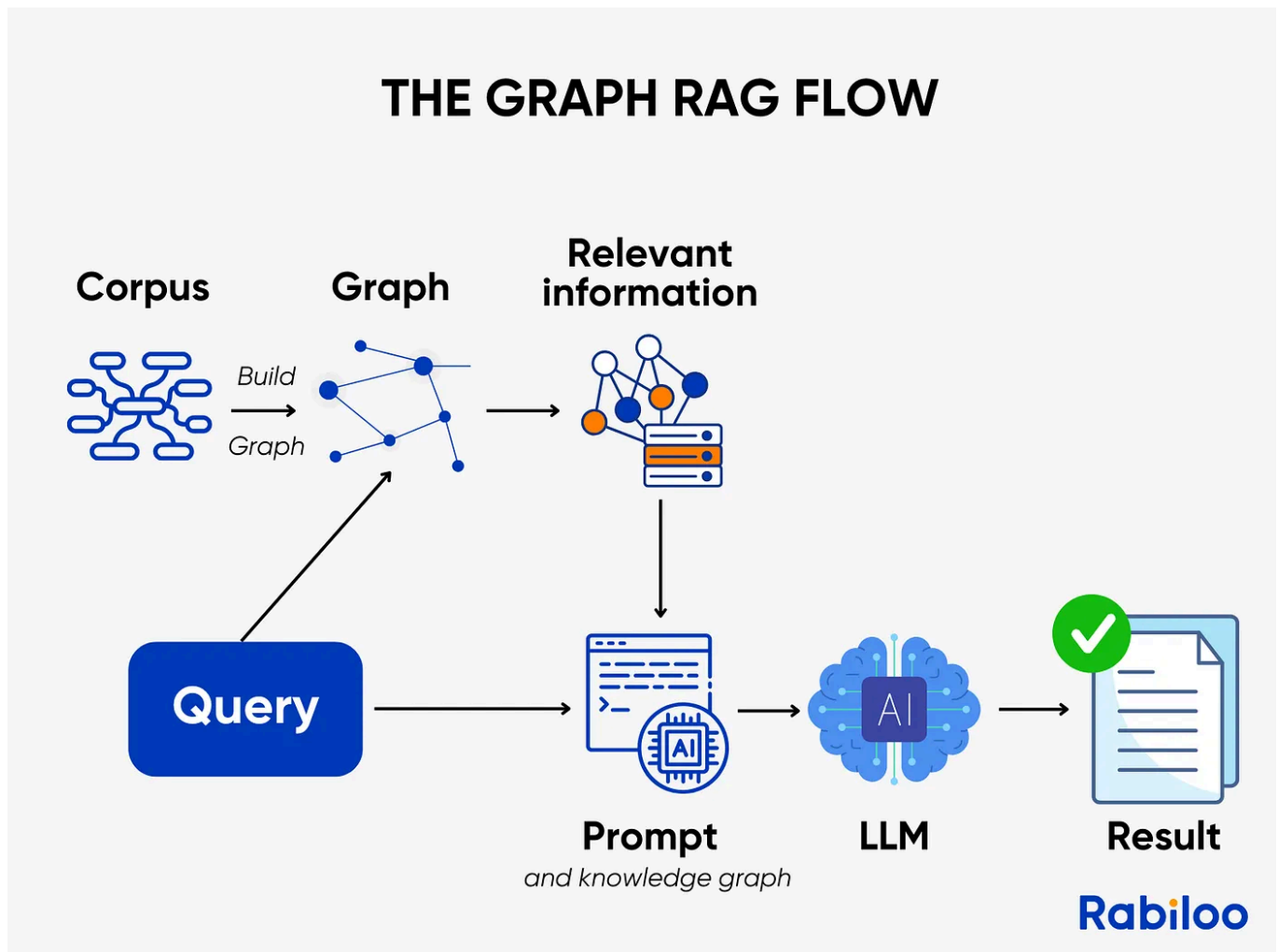
- More adaptable to changing contexts and requirements

Welcome back. You are signed into your member account [jr.....@gmail.com](#).

Cons:

- Higher latency due to multi-step execution
- More expensive to run than simple RAG
- Requires careful tool and agent orchestration
- Overkill for straightforward factual queries

9. GraphRAG: The Relationship Reasoner



Source: <https://rabiloo.com/blog/graph-rag-the-upgrade-that-traditional-rag-needed>

them.

Instead of asking “what text looks similar,” it asks “what is connected, and how?”

How it Works:

1. Graph Construction:

Knowledge is modeled as a graph where nodes are entities (people, organizations, concepts, events) and edges are relationships (affects, depends_on, funded_by, regulated_by).

2. Query Parsing:

The user query is analyzed to identify key entities and relationship types, not just keywords.

3. Graph Traversal:

The system traverses the graph to find meaningful paths that connect the entities across multiple hops.

4. Optional Hybrid Retrieval:

Vector search is often used alongside the graph to ground entities in unstructured text.

5. Generation:

The LLM converts the discovered relationship paths into a structured, explainable answer.

Realistic Example:

Query:

“How do Fed interest rate decisions affect tech startup valuations?”

- Federal Reserve → rate_decision → increased rates
- Increased rates → affects → VC capital availability
- Reduced VC availability → impacts → early-stage valuations
- Tech startups → funded_by → venture capital

The answer emerges from the *relationship chain*, not document similarity.

Why It's Different:

Vector RAG:

“What documents are similar to my query?”

GraphRAG:

“What entities matter, and how do they influence each other?”

This makes GraphRAG far stronger for causal, multi-hop, and deterministic reasoning.

Systems combining GraphRAG with structured taxonomies have achieved accuracy close to 99% in deterministic search tasks.

Pros:

- Excellent at cause-and-effect reasoning
- Highly explainable outputs due to explicit relationships
- Strong performance in structured and rule-heavy domains
- Reduces false positives caused by semantic similarity

- High upfront cost to build and maintain knowledge graphs
- Graph construction can be computationally expensive
- Harder to evolve as domains change
- Overkill for open-ended or conversational queries

How to Actually Choose (The Decision Framework)

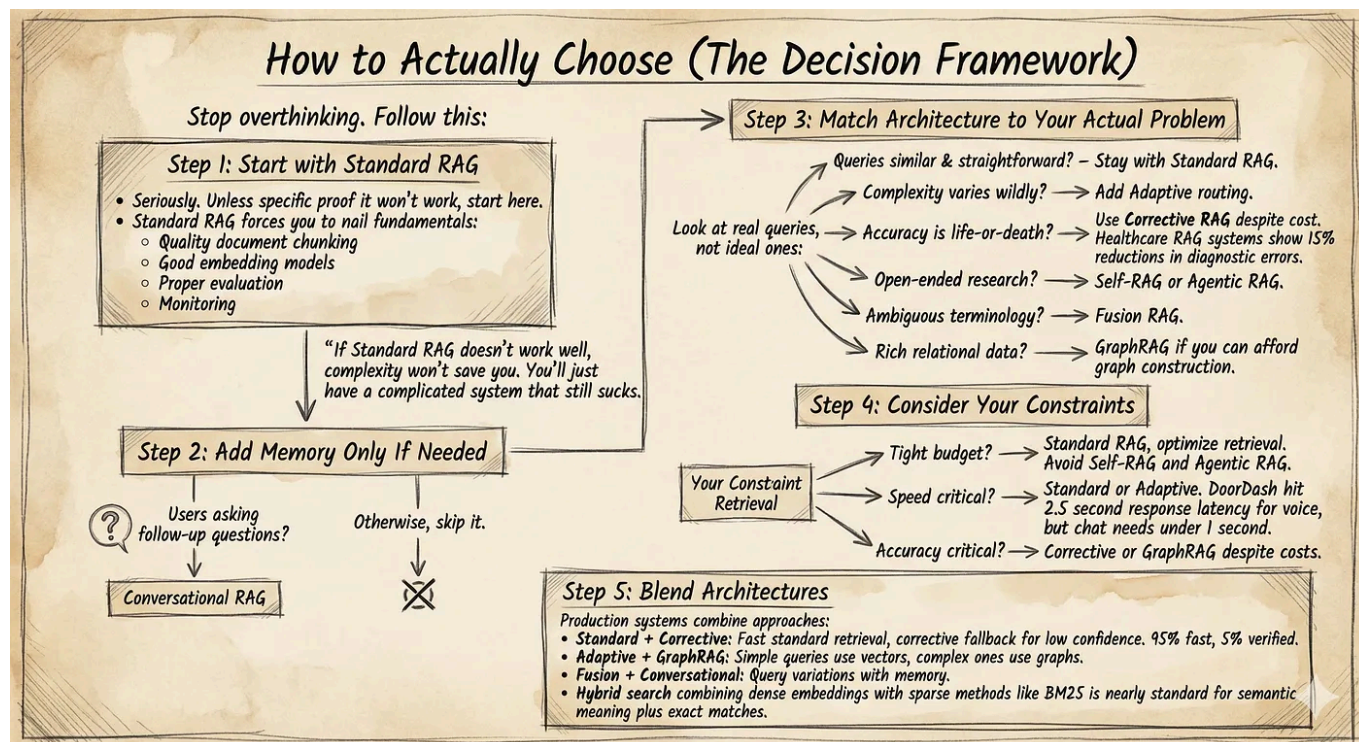


Photo by Gemini

Step 1: Start with Standard RAG

Seriously. Unless you have specific proof it won't work, start here. Standard RAG forces you to nail fundamentals:

- Quality document chunking

- Good embedding models

Welcome back. You are signed into your member account jr*****@gmail.com.

- 
- Monitoring

If Standard RAG doesn't work well, complexity won't save you. You'll just have a complicated system that still sucks.

Step 2: Add Memory Only If Needed

Users asking follow-up questions? Add Conversational RAG. Otherwise, skip it.

Step 3: Match Architecture to Your Actual Problem

Look at real queries, not ideal ones:

- Queries are similar and straightforward? Stay with Standard RAG.
- Complexity varies wildly? Add Adaptive routing.
- Accuracy is life-or-death? Use Corrective RAG despite cost. Healthcare RAG systems show 15% reductions in diagnostic errors.
- Open-ended research? Self-RAG or Agentic RAG.
- Ambiguous terminology? Fusion RAG.
- Rich relational data? GraphRAG if you can afford graph construction.

Step 4: Consider Your Constraints

Tight budget? Standard RAG, optimize retrieval. Avoid Self-RAG and Agentic RAG.

Speed critical? Standard or Adaptive. DoorDash hit 2.5 second response latency for voice, but chat needs under 1 second.

Step 5: **Biena Architectures**

Production systems combine approaches:

- **Standard + Corrective:** Fast standard retrieval, corrective fallback for low confidence. 95% fast, 5% verified.
- **Adaptive + GraphRAG:** Simple queries use vectors, complex ones use graphs.
- **Fusion + Conversational:** Query variations with memory.

Hybrid search combining dense embeddings with sparse methods like BM25 is nearly standard for semantic meaning plus exact matches.

Simple Analogy

Think of an LLM as a **smart employee with a great brain but a terrible memory**.

- **Standard RAG** is like giving them a **file cabinet**. They pull one folder, read it, and answer.
- **Conversational RAG** is the same employee **taking notes during the meeting** so they do not ask the same questions again.
- **Corrective RAG** adds a **senior reviewer** who checks, “Do we actually have proof for this?” before the answer goes out.
- **Adaptive RAG** is a **manager deciding effort level**. Quick reply for easy questions, full research for hard ones.

- Self-RAG is the employee thinking out loud, stopping mid-sentence to

Welcome back. You are signed into your member account jr.....@gmail.com.

- Fusion RAG is asking five coworkers the same question in different ways and trusting what they agree on.
- HyDE is the employee drafting an ideal answer first, then searching for documents that match that explanation.
- Agentic RAG is a team of specialists. Legal, finance, and ops each answer their part, then someone stitches it together.
- GraphRAG is using a whiteboard of relationships instead of documents. Who affects whom, and how.

Red Flags That Kill Projects

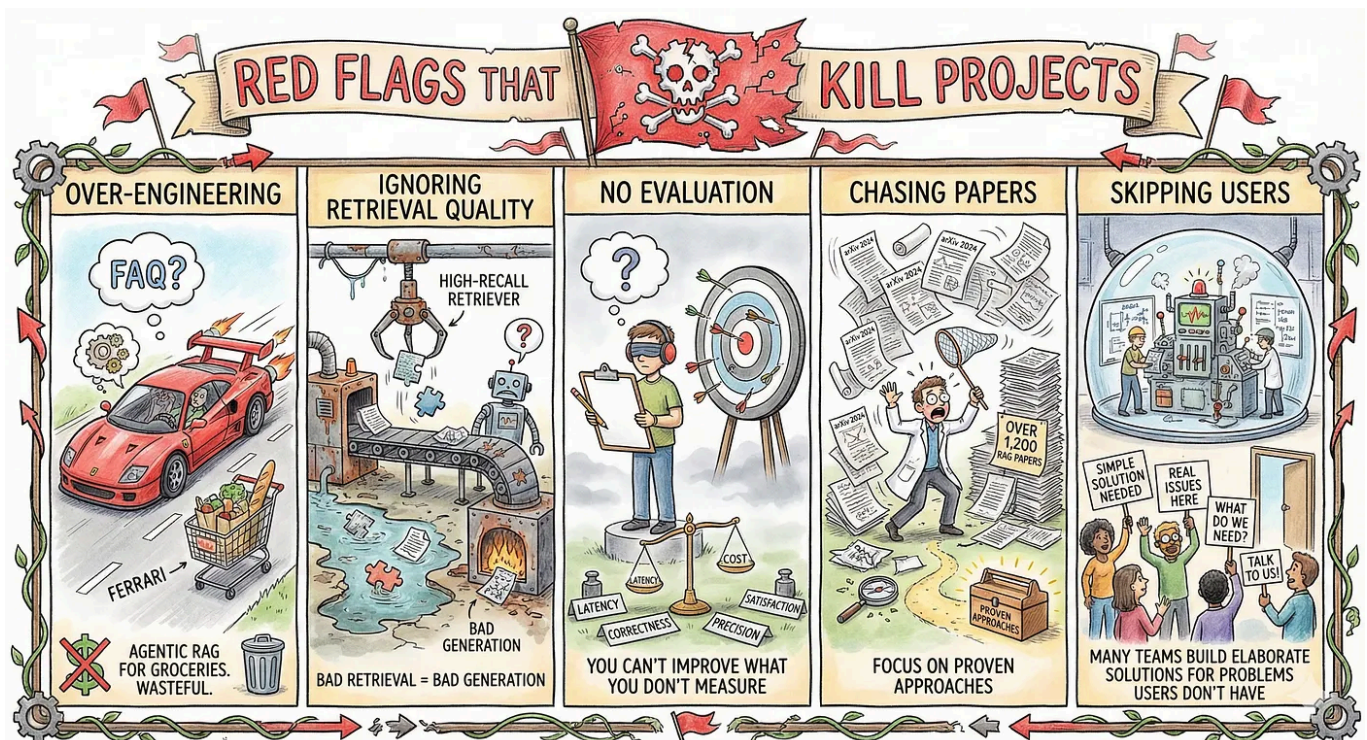


Photo by Gemini

Over-Engineering: Agentic RAG for FAQs is a Ferrari for groceries. Wasteful.

No Evaluation: You can't improve what you don't measure. Track precision, correctness, latency, cost, satisfaction from day one.

Chasing Papers: Over 1,200 RAG papers appeared on arXiv in 2024 alone. You can't implement them all. Focus on proven approaches for your specific problems.

Skipping Users: What do users actually need? Talk to them. Many teams build elaborate solutions for problems users don't have while ignoring real issues.

The Bottom Line

RAG isn't magic. It won't fix bad design or garbage data. But implemented thoughtfully, it transforms language models from confident liars into reliable information systems.

In 2025, RAG serves as the strategic imperative for enterprises, providing the confidence layer needed for businesses to safely adopt generative AI.

The eight architectures solve different problems:

- **Standard:** Fast, simple, start here
- **Conversational:** Adds memory for multi-turn
- **Corrective:** Validates quality, high accuracy
- **Adaptive:** Matches resources to complexity

- **Self-RAG:** Autonomous reasoning, very expensive

Welcome back. You are signed into your member account **jr*****@gmail.com**.

- **HyDE:** Bridges semantic gaps conceptually
- **Agentic:** Orchestrates specialists, most complex
- **GraphRAG:** Relationship reasoning for connected data

The best system isn't the most sophisticated. It's the one that reliably serves your users within your constraints.

Start simple. Measure everything. Scale complexity only with clear evidence it's needed. Master fundamentals first.

.

Artificial Intelligence

Machine Learning

Data Science

Technology

AI



Published in Towards AI

Follow

104K followers · Last published 1 hour ago

Making AI accessible to 100K+ learners. Find the most practical, hands-on and comprehensive AI Engineering and AI for Work certifications at academy.towardsai.net - we have pathways for any experience level. Monthly cohorts still open—use COHORT10 for 10% off!



Written by Divy Yadav

Follow

479 followers · 29 following

I write about LLMs and agentic AI, focusing on why real-world AI systems fail and how to build ones that actually work beyond demos. <https://github.com/dvy246>

Responses (23)



Jesse Reinhalter

What are your thoughts?



Aditya Srivastav

Dec 25, 2025



This is an excellent piece — clear, practical, and obviously written by someone who's actually built systems instead of just reading papers about them.

What really stands out is how grounded this is in production reality. You don't oversell RAG as a... [more](#)



18



1 reply

[Reply](#)



Joana Levtcheva

Jan 4



Great article!



9



1 reply

[Reply](#)



Chris Perrin he/him

Dec 28, 2025



Start simple. Measure everything. Scale complexity only with clear evidence it's needed. Master fundamentals first.

Thanks for this. I need to get better with RAG and this feels like it will help.



8

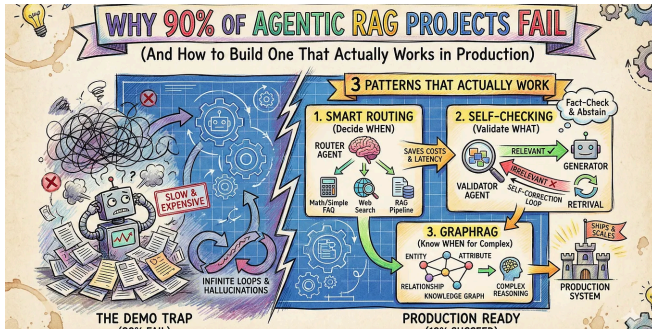


1 reply

[Reply](#)

[See all responses](#)

More from Divy Yadav and Towards AI



 In Towards AI by Divy Yadav

Why 90% of Agentic RAG Projects Fail (And How to Build One That...

A Guide for Developers to Build Production-Level Rag Applications

★ Jan 3 🖱️ 61 💬 3  ⋮

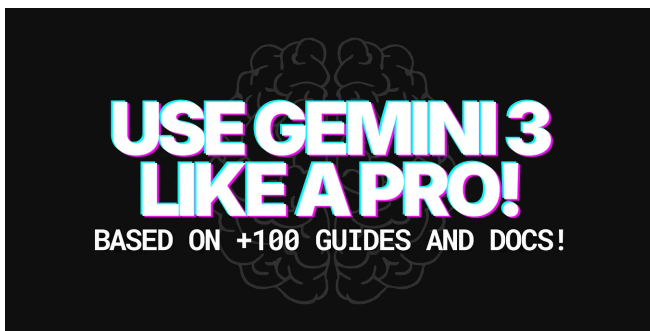


 In Towards AI by Adham Khaled

I Read OpenAI's GPT-5.2 Prompting Guide So You Don't...

(And I turned it into copy/paste templates you can use today.)

★ Dec 14, 2025 🖱️ 1.4K 💬 22  ⋮



 In Towards AI by Adham Khaled

You're Using Gemini 3 Like It's GPT-4. That's Why It's Failing.

Stop fighting the world's smartest model. Here is the new manual for the "Post-Vibe" er...

★ Jan 2 🖱️ 983 💬 27  ⋮



 In Towards AI by Divy Yadav

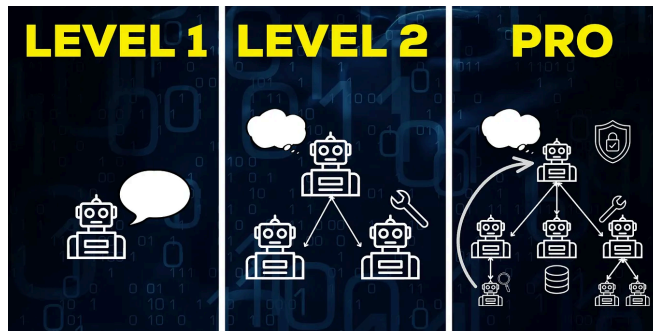
How Video Streaming Actually Works: Why YouTube Starts...

A Detailed system design of streaming platforms like YouTube in simple words

★ Jan 1 🖱️ 144 💬 2  ⋮

Welcome back. You are signed into your member account [jr.....@gmail.com](#).

Recommended from Medium



 In Data Science Coll... by Marina Wyss - Gratitude...

AI Agents: Complete Course

From beginner to intermediate to production.

★ Dec 6, 2025 🖱️ 2.4K 💬 78 📖 + ...

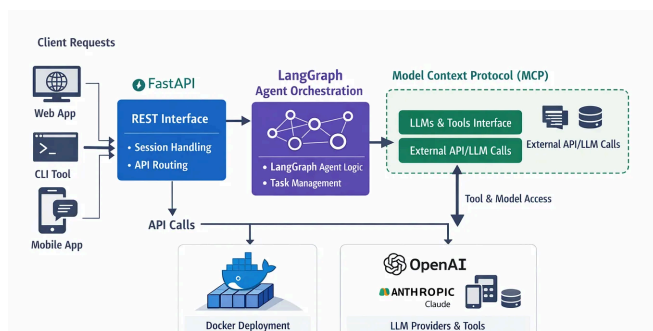


 In AI Advances by Lore Van Oudenhove

Building AI Agents with LangGraph (2026 Edition): A Step-by-Step...

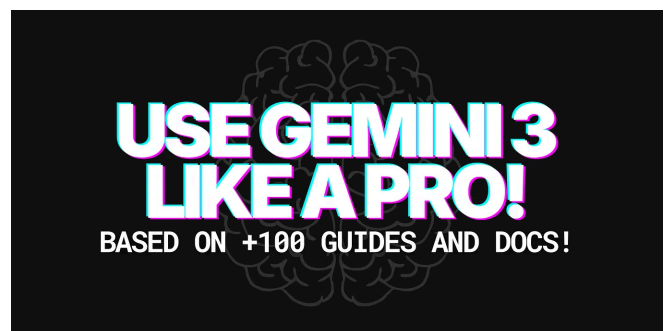
Over the past two years, LangGraph has become the core of almost everything I build...

Jan 3 🖱️ 414 💬 2 📖 + ...



 Shahar Gino

Building Smart Web AI Agents with MCP, LangGraph & FastAPI

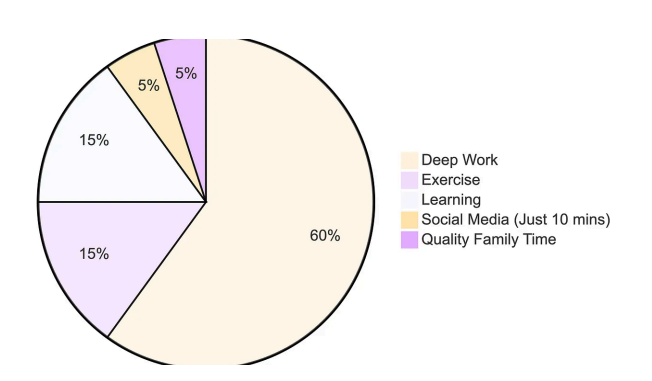


 In Towards AI by Adham Khaled

You're Using Gemini 3 Like It's GPT-4. That's Why It's Failing.

Jan 5👍 197💬 1🔖+⋮

★Jan 2👍 983💬 27🔖+⋮

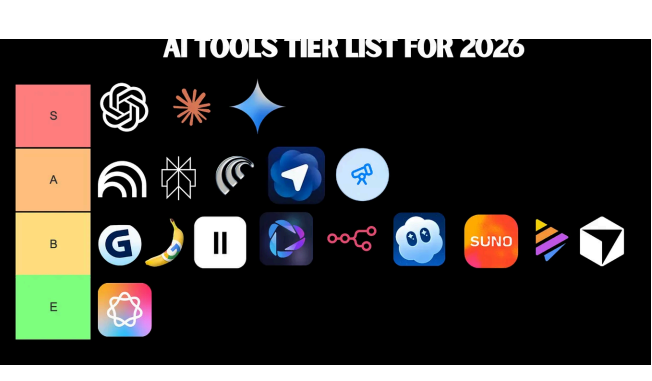


🔗 In Level Up Coding by Teja Kusireddy

I Stopped Using ChatGPT for 30 Days. What Happened to My Brai...

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who...

★Dec 28, 2025👍 940💬 24🔖+⋮



🔗 In Artificial Corner by The PyCoach

The Best AI Tools for 2026

If you're going to learn a new AI tool, make sure it's one of these

★Dec 1, 2025👍 3.9K💬 198🔖+⋮

See more recommendations